



Tecnológico de Monterrey

Actividad Integral 1

Alejandro Sánchez Calderón A01754913

Alan Farid Hernández Sanmartín A01753660

5 de octubre del 2026

Programación de estructuras de datos y algoritmos fundamentales (Gpo 603)

Introducción

Se nos provee un archivo mostrando el tráfico dentro de una empresa que tras ser infectada por un botnet fue utilizada en un ataque DDoS. A través de diversas actividades integradoras procesamos y analizamos dicho archivo buscando resolver algunas preguntas clave: cuál fue el primer host infectado, cuál es la IP del bot master y contra qué dominio se dirigió el DDoS attack.

Lamentablemente, la bitácora provista fue incorrectamente generada por el administrador de la red, por lo que los datos estaban desordenados. Por eso, esta primera actividad se concentra en dos operaciones: ordenar los eventos por fecha y hora, y buscar los eventos de un periodo dado.

Este documento analiza los dos algoritmos que la biblioteca estándar de C++ emplea para esas operaciones, `std::sort` y `std::find_if`, su complejidad temporal y espacial, y las decisiones de diseño que se tomaron al implementarlos.

1. Ordenamiento: `std::sort`

`std::sort` no es un único algoritmo. El estándar de C++ solo exige que la complejidad sea $O(n \log n)$ en el peor caso.

Las implementaciones reales utilizan un algoritmo llamado introsort, que combina tres algoritmos:

Algoritmo	Cuándo se usa	Por qué
Quicksort	Caso general	Muy rápido en la práctica, buena localidad de caché
Heapsort	Si la recursión excede $2 \cdot \log_2(n)$ de profundidad	Garantiza $O(n \log n)$ y evita la degradación a $O(n^2)$
Insertion sort	Subarreglos de menos de 16 elementos	Menor sobrecarga constante que seguir con la recursión

La idea central de introsor empieza con quicksort y analiza la profundidad de recursión. Si detecta que las particiones están quedando desbalanceadas, cambia a heapsort. La elección del pivote es precisamente el punto débil de quicksort.

Complejidad

De media, la distancia entre first y el last es lineal-logarítmica: realiza aproximadamente $N \log_2(N)$ (donde N es la distancia) comparaciones de elementos y hasta esa cantidad se cambia

Para nuestra investigación, n es igual a 33,384, por lo que $\log_2(n)$ es aproximadamente 15.03 y el número de comparaciones está en el orden de $n \log_2(n) \approx 501,700$ comparaciones. Un ordenamiento cuadrático como el bubble sort o selection sort necesitaría del orden de $n^2 \approx 1,114$ millones de operaciones, unas 2,200 veces más trabajo.

2. Búsqueda: `std::find_if`

`std::find_if` realiza una búsqueda lineal o secuencial, recorre el rango `[first, last)` elemento por elemento, evalúa el predicado en cada uno y devuelve un iterador al primer elemento que lo satisface. Si ninguno lo hace, devuelve `last`.

Complejidad

Hasta la distancia linear entre `first` y `last`: llama `pred` para cada elemento hasta que se encuentra un match

3. Complejidad total del programa

Etapa	Operación	Complejidad
1	Lectura y parseo del archivo	$O(n \cdot L)$, L = longitud de línea
2	Ordenamiento (<code>std::sort</code>)	$O(n \log n)$
3	Escritura del archivo ordenado	$O(n \cdot L)$
4	Búsqueda (<code>std::find_if</code>)	$O(n)$
5	Impresión de resultados	$O(k)$, k = eventos mostrados

Complejidad dominante: $O(n \log n)$, determinada por el ordenamiento. Tratando la longitud de línea L como una constante, el analisis y la escritura son $O(n)$.

Complejidad espacial: $O(n)$, pues el vector mantiene los 33,384 eventos en memoria simultáneamente.

Referencias

cplusplus.com. (s.f.-a). *sort*. Recuperado el 5 de octubre de 2026, de

<http://www.cplusplus.com/reference/algorithm/sort/>

cplusplus.com. (s.f.-b). *find_if*. Recuperado el 5 de octubre de 2026, de

https://cplusplus.com/reference/algorithm/find_if/